

PRINCIPIOS DE UI / UX

Guía técnica para programadores: diseño de interfaces, experiencia de usuario y buenas prácticas en código.

Diseño de interfaces

Experiencia de usuario

Buenas prácticas



Dark mode cómodo: contraste alto, colores suaves y menos fatiga visual.

¿Por qué UI/UX importa al programar?

No basta con que el código funcione: debe producir una experiencia clara, accesible y eficiente.



Una app intuitiva reduce errores y dudas del usuario.

El diseño consistente facilita mantenimiento y escalabilidad.

El rendimiento, la accesibilidad y el feedback son parte de la experiencia.

Un programador con bases de UI/UX crea productos más útiles y profesionales.

Flujo de calidad

Código

Funciona

Interfaz

Se
entiende

UX

Se disfruta

Producto

Se
recomienda

Meta técnica: convertir requisitos en interfaces claras, predecibles y rápidas.

“Funciona” no es el final: es el inicio de una buena experiencia.

Los 10 principios heurísticos de Nielsen

Reglas prácticas para detectar problemas comunes de usabilidad antes de entregar una interfaz.

01 Visibilidad

Loaders, barras de progreso y estados.

03 Control

Cancelar, deshacer y confirmar acciones.

05 Prevención

Validación y botones deshabilitados.

07 Eficiencia

Atajos y personalización.

09 Errores claros

Mensajes humanos y solución.

02 Mundo real

Lenguaje familiar, no jerga técnica.

04 Consistencia

Mismos colores, fuentes y patrones.

06 Reconocimiento

Opciones visibles, hints y autocompletado.

08 Minimalismo

Solo información relevante.

10 Ayuda

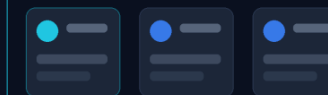
Tooltips, onboarding y documentación.

Jerarquía visual, tipografía y color

La interfaz guía la atención del usuario mediante tamaño, peso, contraste y espaciado.

Ejemplo UI/UX

Jerarquía + contraste + espacios consistentes



Buenas interfaces reducen dudas y errores del usuario

Escala tipográfica

Display

Título

Sección

Cuerpo

Etiqueta

Tokens de color



Primary



Info



Success



Warning

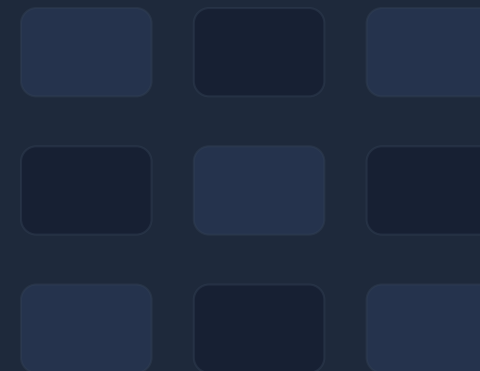


Error



Text

Espaciado y grid



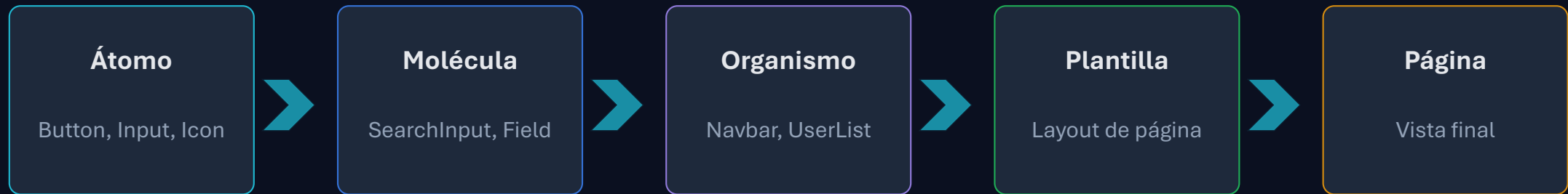
Base de 4px u 8px.
CSS Grid / Flexbox.
Layout responsivo.

Aplicación práctica en una interfaz

- Usa 2 o 3 tamaños principales para que el usuario entienda qué leer primero.
- Mantén colores de estado constantes: éxito, advertencia, error e información.
- El espaciado repetido crea orden visual y evita que la pantalla se vea improvisada.

Atomic Design aplicado a código

Dividir la interfaz en piezas reutilizables mejora orden, escalabilidad y consistencia.

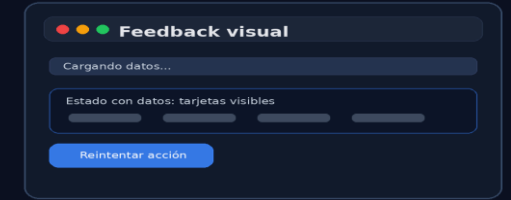


```
const Button = ({ variant, children }) => (  
  <button className={`btn btn--${variant}`}>  
    {children}  
  </button>  
);
```

Componentes aceptan props de configuración.
Evita estilos repetidos o valores “quemados”.
Permite reemplazar piezas sin romper toda la app.

Toda interfaz debe manejar 4 estados

Cargando, error, vacío y con datos: ignorarlos causa pantallas confusas y mala experiencia.



Cargando

Skeletons, spinners o barras de progreso.



Error

Mensaje claro + botón Reintentar.



Vacío

Explicar qué hacer para empezar.



Con datos

Contenido ordenado, filtrable y legible.

Patrón clave en código: si loading → Skeleton; si error → ErrorState; si data vacía → EmptyState; si hay datos → render normal.

Feedback visual: pequeñas señales, gran claridad

Las microinteracciones comunican que una acción fue recibida y evitan incertidumbre.

Botón completo

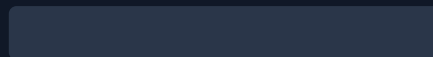
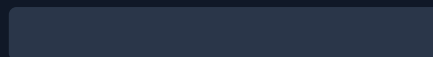
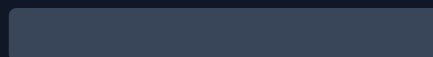
Normal

Hover: sombra + elevación

Focus: outline visible

Disabled: opacidad

Skeleton loader



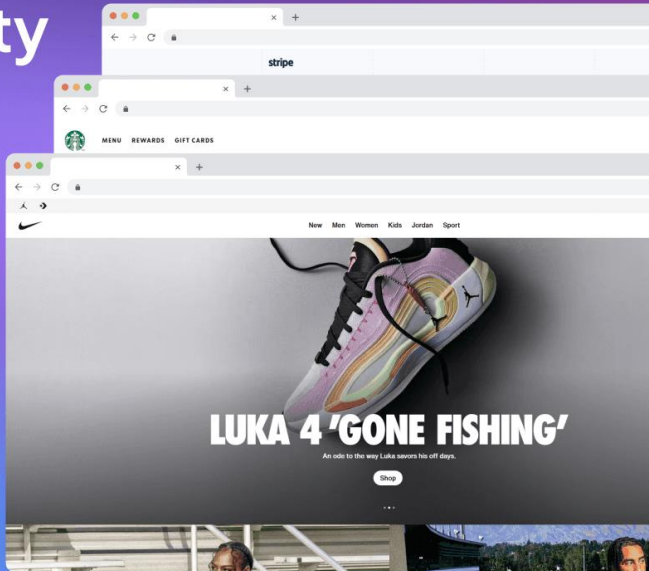
Transiciones cortas: 150–250 ms.
No animar todo: solo lo que aporta significado.
Mantener foco visible para teclado.
Mostrar éxito, error y progreso con claridad.

Accesibilidad web: usable para todos

La accesibilidad mejora la calidad de la interfaz y reduce barreras para teclado, lectores de pantalla y móviles.

Web Accessibility Examples

Get Keyboard
Navigation Right



Reglas esenciales

Usar HTML semántico antes que divs sin significado.

ARIA solo cuando el HTML nativo no alcanza.

Contraste WCAG AA: 4.5:1 en texto normal.

Área mínima táctil: 44 × 44 px.

Nunca eliminar el outline del foco.

Ejemplo: `<nav aria-label="Navegación principal">` mejora la comprensión de lectores de pantalla.

Rendimiento también es UX

Una interfaz lenta se siente rota. Optimizar carga e interacción mejora satisfacción y conversiones.

LCP

< 2.5 s

Preload de imágenes hero, SSR

INP

< 200 ms

Diferir JavaScript no crítico

CLS

< 0.1

Reservar espacio en imágenes

TTFB

< 800 ms

CDN, caching y HTTP/2

```
<img loading='lazy' decoding='async'
width='800' height='450'
alt='Descripción significativa' />
```

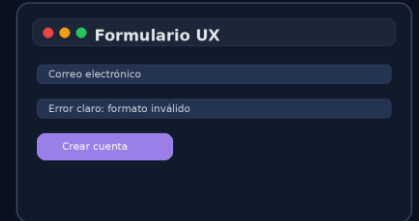
```
const Dashboard = lazy(() =>
import('./Dashboard'));
<Suspense fallback={<PageSkeleton />}>
  <Dashboard />
</Suspense>
```

Idea clave

- Una página rápida reduce abandono y transmite confianza; por eso rendimiento y UX van juntos.
- Optimiza imágenes, divide código pesado y reserva espacio para evitar saltos visuales.

Formularios: donde más fricción aparece

Un formulario bien diseñado reduce errores, acelera el llenado y aumenta la confianza del usuario.



Formulario UX

Correo electrónico

Error claro: formato inválido

Crear cuenta

Principios UX

Una columna por defecto.
Labels siempre visibles.
Validación después del blur.
Agrupar campos con fieldset.
Errores específicos y humanos.
Stepper si hay más de 5 campos.

Ejemplo visual

Nombre

Correo

Formato de email inválido

Contraseña

Crear cuenta

Validación en código

```
if (!value.trim())  
  return 'Campo obligatorio';  
if (!emailRegex.test(value))  
  return 'Correo inválido';
```

```
// feedback inmediato  
if (touched)  
  setError(validate(value));
```

Detalle importante

- Los errores deben aparecer cerca del campo afectado y decir exactamente cómo corregirlos.
- Un formulario simple aumenta la probabilidad de que el usuario lo complete.

Checklist final para Pull Requests

Antes de entregar una interfaz, revisa accesibilidad, rendimiento, UX, funcionalidad y mantenibilidad.

Accesibilidad

✓ alt descriptivo ✓ 44×44 px táctil ✓ contraste WCAG AA ✓
navegación por teclado

Rendimiento

✓ imágenes optimizadas ✓ lazy loading ✓ bundle inicial controlado
✓ sin JS bloqueante

UX y funcionalidad

✓ loading/error/empty/data ✓ validación clara ✓ confirmación
destructiva ✓ funciona en móvil

Código

✓ tokens de diseño ✓ componentes reutilizables ✓ sin estilos inline
innecesarios ✓ CSS organizado

Conclusión: UI/UX no es decoración. Es una práctica técnica que hace que el software sea claro, accesible, rápido y agradable para todos.